Epiplexity and Transfer in Synthetic Time-Series Pretraining

Danila Chernousov Mark Ikonnikov Daniil Kazachkov Alexey Kravatskiy
Moscow Institute of Physics and Technology

21 October 2026

Abstract

Synthetic pretraining offers controllable forecasting tasks, but selecting useful data distributions remains difficult. We investigate whether excess prequential loss, a learning-curve proxy for epiplexity, predicts transfer to real time series. Using `episynth`, we compare eleven synthetic priors at training budgets of approximately 2 and 40 million windows. A probabilistic Moirai transformer is trained from scratch on each prior and evaluated on 55 short-term GIFT-Eval configurations. KernelSynth achieves the best aggregate transfer at both budgets, with geometric mean CRPS ratios to seasonal naive of 0.678 and 0.639. However, excess area has weak rank association with transfer across all eleven priors ($\rho = -0.18$ and 0.00). Stronger associations depend on exploratory subset selection. Endpoint averaging and rare forecast failures substantially affect the measured relationship. Our results identify useful synthetic priors under a fixed forecasting protocol, while showing that excess area alone does not establish a reliable rule for selecting pretraining data.

1 Introduction

Synthetic data make the pretraining distribution an experimental choice. For forecasting, one can generate trends, periodic signals, stochastic innovations, or chaotic trajectories in arbitrary quantities. These distributions differ in both predictability and the structure a model can learn. A low synthetic prediction loss therefore need not imply useful representations for real series. The practical question is which property of a synthetic prior can guide data selection before an expensive downstream evaluation.

Epiplexity describes structure accessible to a computationally bounded observer [7]. This suggests a testable hypothesis: priors from which a forecasting model acquires more structure should yield better transfer. Following our initial study design [3, 4], we examine this hypothesis through excess prequential loss: the accumulated online prediction loss above a held-out endpoint. We compare eleven priors at two data budgets using one architecture and a common evaluation protocol.

Our contributions are threefold. We describe `episynth`, a reproducible pipeline connecting synthetic tasks, probabilistic training, excess-loss estimation, and forecasting evaluation. We analyze 22 primary runs and one initialization repeat, totaling 501,992,448 training windows. Finally, we show that aggregate transfer and excess area have weak full-set association, and identify endpoint choice and forecast tails as material sources of sensitivity.

2 Related work

Finzi et al. [7] motivate prequential epiplexity estimates; Su et al. [12] use epiplexity for language-model data selection and generation. Its forecasting value requires validation: synthetic corpus composition and architecture affect time-series transfer [10]. Chronos introduces KernelSynth data [2]; Moirai supplies our probabilistic transformer [13], and GIFT-Eval the benchmark [1]. Prior-fitted networks motivate approximate Bayesian prediction from synthetic tasks [9]; information-theoretic in-context learning analysis informs the proposed oracle decomposition [8].

3 Methodology

3.1 Problem formulation

Let g denote a synthetic prior over context–target pairs (C, Y), where $Y = (Y_1, \ldots, Y_h)$. A probabilistic forecaster q_θ is trained with loss $\ell_\theta(C, Y) = -\sum_{t=1}^{h} \log q_\theta(Y_t \mid C)$. We ask whether the excess loss accumulated while learning g predicts the model's subsequent forecasting error on real datasets. Model size and training windows are fixed within each comparison; the prior is the experimental variable.

3.2 Synthetic task distributions

The eleven priors cover Gaussian-process kernels (KernelSynth), exponential smoothing (ETS), periodic waveforms, switching stochastic differential equations (SDE), steps, composed trend, seasonality and irregularity (TSI), ARIMA, fractional Brownian motion (fBm), GARCH volatility, chaotic dynamics, and TimeSynth signals. A seeded generator produces a length-1024 series. A separate seeded procedure selects a start, context C of length c, and a future target Y of horizon $h = 64$. The saved configurations sample 492 context lengths between 13 and 512, with weights derived from GIFT-Eval history lengths. Short contexts are padded and masked; future values are excluded from context normalization. Uniform dequantization of width $10^{-3}\sqrt{\widehat{\mathrm{Var}}(C) + 10^{-5}}$ smooths the observations. The same series seed produces the same window independently of batch composition or worker count.

3.3 Probabilistic model and training

The observer is a Moirai 1.x port trained from scratch: six transformer layers, width 384, patch size 32, and 10,746,600 parameters. Its density head mixes Student-t, fixed-scale Gaussian (scale 10^{-3}), and log-normal components, with context scaling. Table 1 gives the completed budgets.

Table 1: Training series. Counts are per prior; the extra seed belongs to series 2.

Series	Windows N	Optimizer steps	Training data access
1 (2M)	1,999,872	1,953	Precomputed corpora
2 (40M)	39,999,488	39,062	Online; KernelSynth precomputed

Both series use batch size $B = 1024$, AdamW with learning rate 10^{-3}, 200 warmup steps, weight decay 0.1, $\beta = (0.9, 0.98)$, $\epsilon = 10^{-6}$, and gradient clipping at 1. The EMA time constant is 2% of the total step budget. Training uses bfloat16. At 1, 2, 5, 10, 20, 50, and 100% of the budget, raw and EMA weights are scored on 100,000 held-out windows, starting at seed 10^9. Additional float32 scores are saved; the area below uses the matching bfloat16 floor. Primary runs share initialization seed 0 and training seeds starting at 0; Waveform additionally uses initialization seed 1. The two budgets therefore share seed ranges and are not independent replications. Windows and model size are matched within a series; hardware and data-generation costs vary.

3.4 Excess prequential loss as a structural proxy

For time-bounded probabilistic programs $\mathcal{P}_T$, formal epiplexity is the model-description part of an optimal two-part code [7]:

$$P_T^\star \in \arg\min_{P \in \mathcal{P}_T} \{|P| + \mathbb{E}[-\log_2 P(Y \mid C)]\}, \qquad S_T(Y \mid C) = |P_T^\star|. \tag{1}$$

The programs must both sample and evaluate probabilities within the time bound. The experiment estimates an algorithm-dependent learning-curve proxy. Let u_k be the mean negative log density per target point in batch k, measured *before* its update, and $\widehat{\ell}_N^{\mathrm{EMA}}$ the final held-out EMA loss in nats per window. The signed excess area is

$$A_N = Bh \sum_{k=1}^{K} u_k - N\widehat{\ell}_N^{\mathrm{EMA}}, \qquad \widehat{S}_N = A_N / \ln 2, \qquad N = BK. \tag{2}$$

All reported areas use million nats; division by $\ln 2$ yields bits. Figure 1 shows the underlying curves. A common conditional-entropy term cancels in the expected excess loss: for fresh independent tasks and an unbiased endpoint estimate, it becomes a sum of differences between the online and endpoint conditional KL divergences to the true prior predictive. This motivates measuring learning progress, but does not identify the optimal program in the formal definition.

The floor matters. Online losses use raw weights, whereas the primary endpoint uses EMA weights. Relative to a raw endpoint, this adds $N(\widehat{\ell}_N^{\mathrm{raw}} - \widehat{\ell}_N^{\mathrm{EMA}})$ to the area. A floor error δ shifts A_N by $-N\delta$; positive clipping would define another statistic. Continuous-density losses also require consistent units and quantization to admit a coding interpretation. We retain the repository's original-unit densities and its signed area throughout.

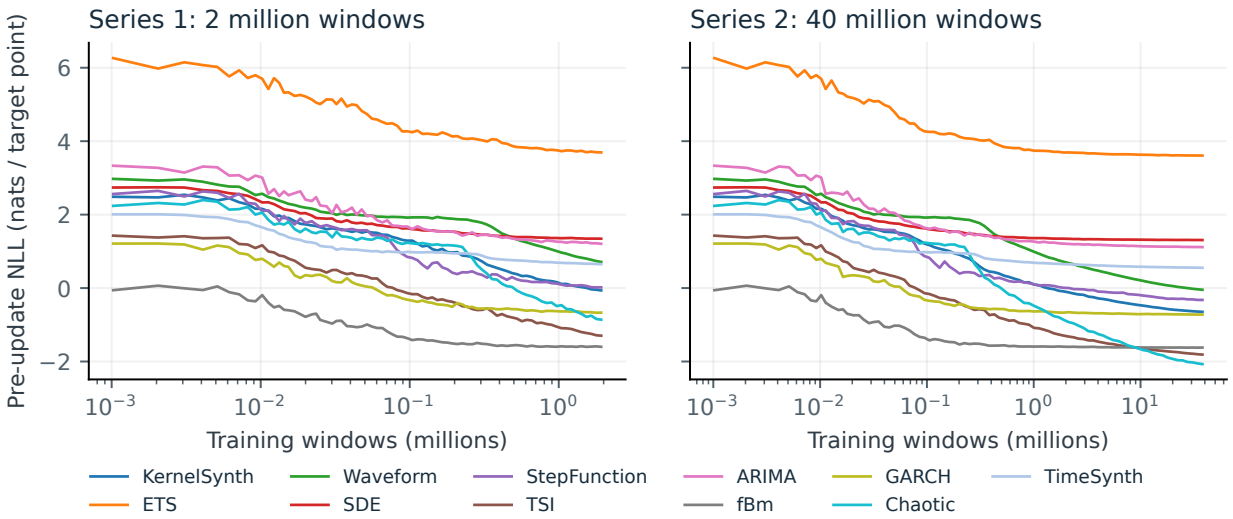


Figure 1: Saved pre-update training losses for the eleven primary runs. Curves average log-spaced groups of steps for display; Eq. (2) uses every exported step. Negative continuous-density losses are valid.

3.5 Transfer metrics and uncertainty

The final EMA model is evaluated without fine-tuning on 55 short-term GIFT-Eval configurations. For metric m, we normalize by the saved seasonal-naive baseline and weight each configuration equally:

$$R_g^{(m)} = \exp\left\{\frac{1}{55}\sum_{j=1}^{55}\log\frac{m_{g,j}}{m_{\text{naive},j}}\right\}. \tag{3}$$

Smaller ratios are better; $R < 1$ beats the baseline. The main metric is the sample-based `scaled_crps`; the separate nine-quantile `mean_weighted_sum_quantile_loss` field is not substituted for it. MASE, median ratios, and average configuration ranks are sensitivity checks. We compute Spearman correlations across the eleven generator families and 95% percentile intervals from 10,000 paired bootstrap resamples of families (seed 0). These intervals describe variation across the small set of priors, not training seed uncertainty. The 55 configurations include related datasets/frequencies and are not 55 independent training replications.

4 Implementation

The measurement layer currently implements prequential coding through `estimators.build("prequential")`. For the deployable library we propose four measurement methods: prequential coding, requential coding [11], predictive $\mathcal{V}$-information [14], and Bayesian predictive log-loss using the existing OU reference. Appendix B states each method's scientific purpose, inputs, computation, outputs, and readiness. Prequential coding is implemented; requential is a registered stub; $\mathcal{V}$-information requires implementation; the OU reference is implemented under `checks/` and needs a public measurement wrapper. Only prequential experiments enter the transfer results reported here.

These methods use shared infrastructure: seeded context–target generation, AdamW/EMA training, checkpointing, and GIFT-Eval evaluation. Generation supplies tasks, training fits a predictor and records its trajectory, and evaluation measures downstream forecast error. The estimator specifies which information quantity is computed from these artifacts. Current interfaces, proposed method contracts, and examples are documented in the repository's library guide. Source and experiment outputs are available in the project repository.

5 Experimental evaluation

We use the completed training series of 19–20 October 2026 [6]. All 23 runs are included; the additional Waveform initialization is reported separately from the eleven-family comparisons. Statistics are recomputed from the archived loss curves, held-out checkpoints, and forecast summaries.

5.1 Training budget and downstream performance

Table 2 reports all primary runs and the additional initialization. KernelSynth achieves the lowest aggregate CRPS ratio at both budgets and beats seasonal naive in 50 of 55 configurations in each series. ETS ranks second. Increasing the budget improves aggregate CRPS for eight of eleven primary priors; Waveform, fBm, and Chaotic worsen. StepFunction improves substantially, from 1.607 to 0.839. Thus more optimization helps many priors, but does not uniformly improve transfer.

Table 2: Recomputed signed EMA-floor area A (million nats) and geometric mean metric ratios to seasonal naive. Lower metric ratios are better. Seed 0 unless specified; the seed-1 repeat is excluded from the eleven-family correlations.

	Series 1: 2M		Series 2: 40M		
Prior	A	CRPS	A	CRPS	MASE
KernelSynth	68.4	**0.678**	926.1	**0.639**	0.828
ETS	26.7	0.731	138.6	0.717	0.941
Waveform	70.9	0.818	1061.2	1.373	0.986
SDE	11.0	0.872	34.6	0.861	1.142
StepFunction	49.5	1.607	588.3	0.839	1.092
TSI	64.8	0.803	765.9	0.776	1.028
ARIMA	24.2	0.847	154.5	0.833	1.089
fBm	9.6	0.853	40.6	0.867	1.110
GARCH	16.6	0.879	100.9	0.874	1.116
Chaotic	126.2	1.171	1956.6	1.285	1.616
TimeSynth	15.3	0.974	114.9	0.851	1.123
Waveform, seed 1	—	—	1063.6	0.972	0.995

5.2 Does excess area explain transfer?

Across all eleven priors, the association between area and aggregate CRPS is weak: $\rho = -0.18$ (95% interval $[-0.81, 0.51]$) at 2M and $\rho = 0.00$ ($[-0.89, 0.61]$) at 40M (Figure 2). The intended relationship, larger learned structure predicting lower error, would yield a negative correlation. Final EMA loss gives $\rho = -0.13$ and -0.22, respectively; these results do not demonstrate that area predicts transfer better than final loss.

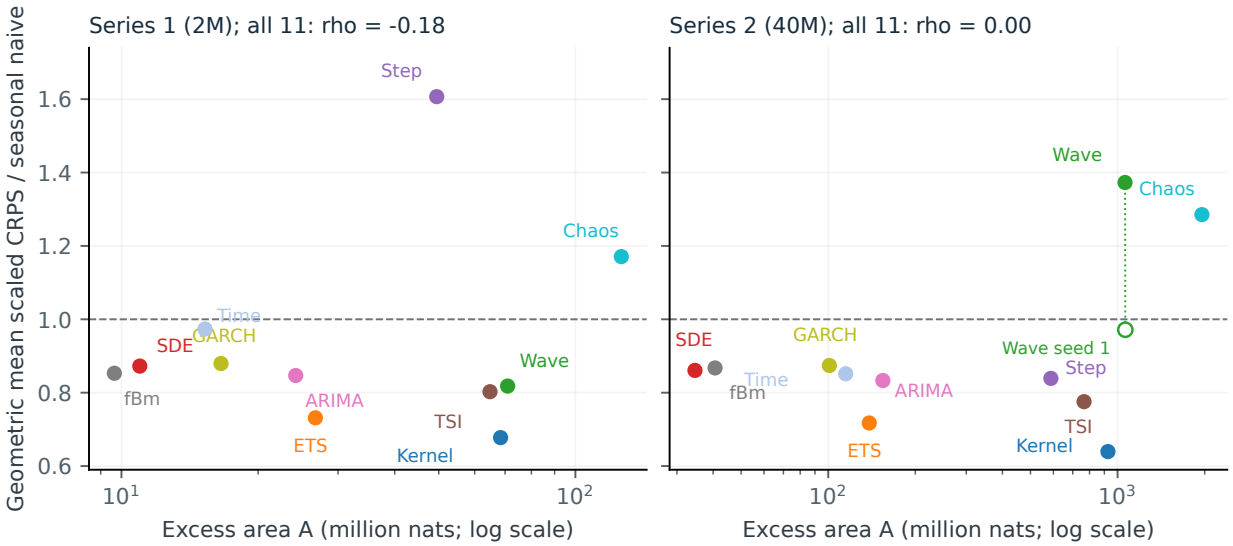


Figure 2: Area and transfer for every prior. The horizontal line is seasonal naive. Waveform's extra initialization (open marker) changes aggregate CRPS markedly despite an almost identical area. Horizontal axes are logarithmic.

Selected subsets show stronger associations. Excluding Chaotic and StepFunction gives $\rho = -0.73$ at 2M ($n = 9$, interval $[-0.95, -0.14]$), but only -0.28 at 40M. Additionally excluding Waveform at 40M gives -0.83 ($n = 8$, $[-1.00, -0.24]$). The sensitivity analysis also explores fifteen area variants. Such exclusions and metric choices are exploratory, selected after seeing the results; they do not establish a universal ranking rule.

Waveform illustrates sensitivity to forecast tails. At 40M, a second initialization gives almost identical areas

(1061.2 versus 1063.6 million nats) but CRPS ratios of 1.373 and 0.972. The median configuration ratios are much closer (0.750 and 0.721). The worst seed-0 configuration, `bitbrains_rnd/5T/short`, has a CRPS ratio of about 2.67×10^5; the same configuration gives 58.0 for seed 1. The saved summaries locate the failure but do not expose its individual forecast samples. Replacing Waveform seed 0 with seed 1 still leaves the full-set area correlation near zero ($\rho = 0.009$).

The floor is a second sensitivity: ARIMA's 40M area falls from 154.5 to 15.4 million nats when using the raw endpoint. Full-set raw-floor correlations are -0.15 and $+0.19$. An EMA endpoint therefore changes the measured amount of structure substantially, without resolving the lack of a stable full-set association.

5.3 Short prefixes and a compression baseline

The seven held-out checkpoints permit a first look at early prediction. At each prefix, we recompute area using that checkpoint's own EMA floor, and compare its ranking with final downstream CRPS (Figure 3). We also compare checkpoint loss and the loss drop from the mean loss in the initial 1% of training steps. No future endpoint loss enters a prefix estimate. Associations vary with budget and checkpoint; this is an exploratory correlation analysis, without a predictor trained and validated on unseen generator families.

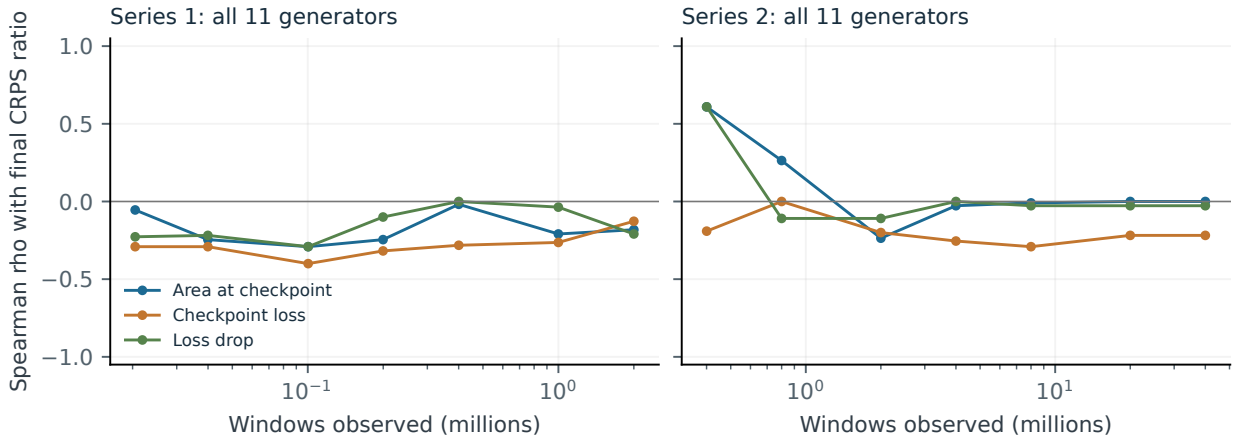


Figure 3: Prefix statistics versus final aggregate CRPS across all eleven priors. Negative values favor larger predictor values. Checkpoints use their own floor; the curves do not constitute an out-of-sample prediction test.

The gzip baseline uses 5,000 series for each of ten priors (TimeSynth absent), with raw float32 and range-normalized int16 rates. Int16-rate correlations with CRPS are -0.59 at 2M and -0.47 at 40M; both intervals include zero. Current uncertainty establishes no advantage for either proxy.

6 Discussion and conclusion

KernelSynth and ETS provide the strongest aggregate transfer in our comparison. Excess prequential loss, however, does not consistently rank all eleven priors. This limits its present use as a data-selection criterion: a large area measures learning progress on the synthetic task, which need not capture the structure needed by the downstream benchmark. Moreover, Eq. (2) makes area a function of accumulated loss and the endpoint; it is not an independent source of information once both are known.

The study uses one model configuration, mostly one initialization per prior, and matched data budgets rather than measured equal-compute budgets. Subset correlations and prefix analyses are exploratory. These constraints preclude claims of architecture-independent epiplexity or a validated early transfer predictor. Future studies should predeclare the floor and aggregation protocol, repeat initializations, and validate prior selection on held-out generator families. Oracle, noise-replacement, mixture, and model-size studies remain future work. The current evidence supports useful prior choices under a fixed protocol, but does not establish a general epiplexity-based ranking rule.

References

[1] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. GIFT-Eval: A benchmark for general time series forecasting model evaluation. arXiv:2410.10393, 2024. URL https://arxiv.org/abs/2410.10393.

[2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2403.07815.

[3] Danila Chernousov. Epiplexity of synthetic data and the quality of time series models. Research proposal 3.4, draft dated 15 October; supplied `proposal_3.4.pdf`, 2 pages, in Russian; title translated, 2026.

[4] Danila Chernousov. Epiplexity of synthetic data and time series models: Literature review. Supplied slides, `lit_talk_v2.pdf`, 14 pages, in Russian; title translated, 2026.

[5] Chernousovdv. Add episynth: Synthetic generators, task windows, Moirai port, training with per-window loss, GIFT-Eval evaluation. Epiplexity repository, commit `5b350cd`, 19 October, 2026. URL https://github.com/intsystems/Epiplexity/commit/5b350cdbb9e4c74efb3d351b6b1243e051a03f76.

[6] Chernousovdv. Saved outputs of episynth series 1 and 2: Synthetic pretraining and GIFT-Eval evaluation. Epiplexity repository, commit `7d2b001`, 21 October; runs dated 19–20 October, 2026. URL https://github.com/intsystems/Epiplexity/commit/7d2b001d972169efdb19e3c8e256a182ba4c09f8.

[7] Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, and Andrew Gordon Wilson. From entropy to epiplexity: Rethinking information for computationally bounded intelligence. arXiv:2601.03220, 2026. URL https://arxiv.org/abs/2601.03220.

[8] Hong Jun Jeon, Jason D. Lee, Qi Lei, and Benjamin Van Roy. An information-theoretic analysis of in-context learning. In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2401.15530.

[9] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2112.10510.

[10] Aaryan Nagpal, Debdeep Sanyal, Murari Mandal, Dhruv Kumar, and Saurabh Deshpande. Mix, don't pick: Why synthetic corpus composition matters for time series foundation model pretraining. arXiv:2606.09912, 2026. URL https://arxiv.org/abs/2606.09912.

[11] Shikai Qiu, Marc Finzi, Yujia Zheng, Kun Zhang, and Andrew Gordon Wilson. Requential coding: Pushing the limits of model compression with self-generated training data. arXiv:2607.11883, 2026. URL https://arxiv.org/abs/2607.11883.

[12] Ellen Su, Andres Potapczynski, Shikai Qiu, Edward Hughes, and Andrew Gordon Wilson. Epiplexity guided data selection and generation for out-of-distribution generalization. arXiv:2608.11746, 2026. URL https://arxiv.org/abs/2608.11746.

[13] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2402.02592.

[14] Yilun Xu, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. A theory of usable information under computational constraints. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/2002.10689.

A Evidence and reproducibility

Danila Chernousov (`Chernousovdv`) conducted the archived training; this article analyzes the shared artifacts without rerunning training. Inputs at commit `7d2b001` are configurations, training curves, held-out logs and evaluation summaries under `experiments/episynth/`, in `series1_2026-10-19/` and `series2_2026-10-19/`. Scripts hash every input. Training used RTX 4090/A100 hardware; KernelSynth's long series used a GPU-generated float64 corpus. Its interrupted first checkpoint was recovered from saved weights; exported steps are contiguous.

Batch means reconstruct the linear signed area. Six-significant-digit exports imply a worst-case rounding bound below 0.013 million nats. All 22 primary areas match the uploaded per-window analysis to its 0.1-million-nat precision. Release weights and large per-window loss vectors were not downloaded; clipped variants, held-out standard errors and individual forecast failures are consequently not reconstructed. Bootstrap intervals have no adjustment for exploratory comparisons.

Seasonal-naive point metrics match the original leaderboard on all 55 configurations, with maximum relative discrepancy below 1.7×10^{-7} across MASE, sMAPE, ND, MAE and MSE. Report-time checks of `tests/episynth` give 72 passed, 2 skipped and 0 failed; the skips need external GIFT-Eval data and the original Moirai checkpoint. The OU Bayesian reference is covered by these checks; no archived OU training/transfer run is present. The earlier Moirai-port agreement claim (7×10^{-4} nats/point) comes from the author's implementation commit [5], not an independent remeasurement.

For scaling $z = (y - \mu)/s$, $\mathrm{NLL}_{\mathrm{raw}} = \mathrm{NLL}_z + h \log s$: normalized NLL subtracts $h \log s$, contrary to an existing training docstring. This analysis retains raw-unit scores. `analyze_runs.py` rebuilds statistics/tables, `make_plots.py` figures, and `build_report.py` the checked PDF. `reproduce.py` also records the environment and reruns checks. Evidence is in `data/`; versions are pinned in `requirements.txt`.

B Four measurement methods and their library contracts

These four measurement methods define the proposed library scope. Generation, optimization and forecast scoring supply shared infrastructure; statuses describe current code.

1. **Prequential coding with a held-out-floor decomposition (implemented).** *Purpose:* measure the predictive cost of the actual training sequence and separate the final predictor's residual cost from the excess incurred while learning. *Input:* ordered context–target windows, initial model, update rule and budget; for recomputation, pre-update losses ℓ_i and M held-out endpoint losses ℓ_j^{H}, all in nats per window. With N training windows,

$$L_{\mathrm{pre}} = \frac{\sum_{i=1}^{N} \ell_i}{\ln 2}, \quad \widehat{H} = \frac{N}{M \ln 2} \sum_{j=1}^{M} \ell_j^{\mathrm{H}}, \quad \widehat{S} = L_{\mathrm{pre}} - \widehat{H}. \tag{4}$$

   Score each batch before its update, then train; evaluate the selected final raw/EMA predictor on held-out seeds and compute the decomposition. *Output:* `total_bits` $= L_{\mathrm{pre}}$, `data_part_bits` $= \widehat{H}$, `model_part_bits` $= \widehat{S}$, and `n_windows` $= N$. $\widehat{S}$ is a signed, training-dependent structural proxy. Continuous-density log-loss requires quantization and a codec to yield a literal bitstream. Code: `PrequentialEstimator.run/from_run_dir` in `estimators/prequential.py`; checks: `test_estimators.py`, `test_analysis.py`.

2. **Requential coding (registered stub; implementation required).** *Purpose:* describe a student's learned predictive behavior by transmitting teacher-selected synthetic training samples, with cost driven by disagreement. *Input:* teacher and student predictors, real-data teacher updates, shared contexts, initialization/update rules, and reproducible proposal randomness. Relative entropy coding selects a student proposal with the teacher's distribution; the decoder regenerates that proposal from its index and repeats the student update. For whole-message teacher/student distributions Q_k, P_k and KL measured in bits,

$$D_k = \mathbb{E}_{Y \sim Q_k} \log_2 \frac{Q_k(Y)}{P_k(Y)}, \qquad \mathbb{E}L_{\mathrm{req}} \le C_{\mathrm{setup}} + \sum_k \mathbb{E}[D_k + 2\log_2(1 + D_k) + \kappa]. \tag{5}$$

   Here $\kappa < 5.21$; the outer expectations include trajectory randomness. *Required output:* student checkpoint, cumulative KL estimate and coding-overhead bound, with units and shared-side-information assumptions. Distinguish measured KL, a code-length bound, and a realized bitstream. Seeded sampling and projection/decoding rules remain to be implemented. `RequentialEstimator.run` in `estimators/requential.py` raises `NotImplementedError`; registration alone does not make it runnable.

3. **Predictive $\mathcal{V}$-information (proposed; implementation required).** *Purpose:* measure how much observing context improves prediction of the future for a specified predictive family $\mathcal{V}$. Define

$$H_{\mathcal{V}}(Y \mid X) = \inf_{q \in \mathcal{V}} \mathbb{E}[-\log_2 q(Y \mid X)], \qquad I_{\mathcal{V}}(C \to Y) = H_{\mathcal{V}}(Y \mid \varnothing) - H_{\mathcal{V}}(Y \mid C). \tag{6}$$

   Input: paired contexts/targets, a predictive family permitting context to be ignored, common train/validation/test splits, and fitting budgets. Fit a context-free predictor $q_0(Y)$ and a conditional predictor $q_1(Y \mid C)$; score both on the same untouched M test windows:

$$\widehat{I}_{\mathcal{V}} = \frac{1}{M} \sum_{j=1}^{M} \log_2 \frac{q_1(Y_j \mid C_j)}{q_0(Y_j)}. \tag{7}$$

   Required output: both test cross-entropies and their gain in bits per window, with family, fitting and split metadata. Larger gain means more usable context information. Finite fitting/test error can give a negative estimate; retain its sign and report fitting error as a limitation to approximating the population optima. This method needs its own predictive-gain result contract.

4. **Bayesian predictive log-loss by quadrature (OU reference implemented).** *Purpose:* calibrate forecast loss against a known generative prior and separate fitting error from the cost of factorizing future predictions. *Input:* OU context–target windows (horizon h) and quadrature nodes φ_j for the known AR(1) prior, $\varphi = \sin u$ with uniform u. For J equal-weight nodes,

$$M_J(x) = \frac{1}{J} \sum_{j=1}^{J} p_{\varphi_j}(x), \quad w_j(C) = \frac{p_{\varphi_j}(C)}{\sum_r p_{\varphi_r}(C)}, \quad \ell_{\mathrm{joint}} = -\log \frac{M_J(C, Y)}{M_J(C)}, \tag{8}$$

$$\ell_{\mathrm{point}} = -\sum_{t=1}^{h} \log \left[\sum_{j=1}^{J} w_j(C)\, \mathcal{N}(Y_t; \varphi_j^t C_c, 1 - \varphi_j^{2t}) \right]. \tag{9}$$

   C_c is the last context value. The joint score uses the chain rule; the pointwise score conditions only on context, matching Moirai's factorization. *Output:* two per-window NLL arrays in nats; their means divided by h give nats/point. Compare model NLL to the pointwise reference on the same windows. Code: `checks.ou_bayes.exact_window_nll/pointwise_window_nll`; checks: `test_ou_bayes.py`. Verify quadrature convergence and use clean OU windows. A common public-method wrapper remains proposed.